



CSE445 Report

**Project Title: Phishing URL Detection Using Machine Learning,
Contextual Models, and Paper-Inspired Pipelines**

Sakia Tabassum Ananna	ID # 2221433042
Md. Asraful Hossain Riyan	ID # 2232845642
Md. Mahfujur Rahman	ID # 2311858042

Group Number: 06
Semester: Summer 2026

Faculty:
Riasat Khan,
Assistant Professor,
ECE Department

Individual Contribution Table

Section	Contributing Member Name	
IEEE Paper Formatting	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman	
Grammarly Check	Md. Mahfujur Rahman	Grammarly Score: 95
Abstract	Sakia Tabassum Ananna, MD, Mahfujur Rahman	
Keywords	Sakia Tabassum Ananna, MD, Mahfujur Rahman	
Introduction Motivation	Md. Asraful Hossain Riyan, Sakia Tabassum Ananna	
Paper Review 1: Barik et al.	Md. Asraful Hossain Riyan	[4]
Paper Review 2: Duarte et al.	Sakia Tabassum Ananna	[5]
Paper Review 3: Bonet et al.	Md. Mahfujur Rahman	[6]
Dataset and Preprocessing	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman, Sakia Tabassum Ananna	
Logistic Regression	Md. Asraful Hossain Riyan	
Support Vector Machine	Sakia Tabassum Ananna	
Decision Tree	Md. Mahfujur Rahman	
Random Forest	Md. Asraful Hossain Riyan	
XGBoost	Md. Mahfujur Rahman	
Hyperparameter Tuning and Model Selection	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman, Sakia Tabassum Ananna	
Stacking and Blending	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman	
Random Oversampling	Md. Asraful Hossain Riyan	
SMOTE	Sakia Tabassum Ananna	
ADASYN	Md. Mahfujur Rahman	
TF-IDF	Sakia Tabassum Ananna	
Word2Vec	Md. Asraful Hossain Riyan	
FastText	Md. Mahfujur Rahman	
BERT	Sakia Tabassum Ananna	
DistilGPT2	Md. Asraful Hossain Riyan	
Qwen	Md. Mahfujur Rahman	
WSADBench-Inspired Pipeline	Md. Asraful Hossain Riyan	Individual Pipeline
StablePFN-Inspired Pipeline	Sakia Tabassum Ananna	Individual Pipeline
Pretrained iLTM Pipeline	Md. Mahfujur Rahman	Individual Pipeline
LIME Explainability	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman	
Feature-Group Ablation	Md. Asraful Hossain Riyan, Sakia Tabassum Ananna	
Results and Discussion	Md. Asraful Hossain Riyan, Md. Mahfujur Rahman, Sakia Tabassum Ananna	
Figures, Tables, and Equations	Md. Asraful Hossain Riyan, Sakia Tabassum Ananna, Md. Mahfujur Rahman	
Conclusion and Final Report Integration	Sakia Tabassum Ananna, Md. Mahfujur Rahman	
References	Sakia Tabassum Ananna	
Source Code and Reproducibility	Md. Mahfujur Rahman	

Phishing URL Detection Using Machine Learning, Contextual Models, and Paper-Inspired Pipelines

MD. Asraful Hossain Riyan

Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
asraful.riyan@northsouth.edu

Sakia Tabassum Ananna

Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
sakia.ananna@northsouth.edu

Md. Mahfujur Rahman

Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
mahfujur.rahman1@northsouth.edu

Abstract—Phishing URLs trick users into giving away passwords and financial data. This study tests phishing URL detection using sixteen numerical features, covering lexical, structural, domain, and security traits. After data cleaning, exactly thirty-one thousand eight hundred eight records were left for training and testing. The tests included five classic machine learning models, Stacking and Blending, three data augmentation ways, three NLP methods, three contextual models, and three paper-inspired pipelines. Among standard classifiers, XGBoost performed best, hitting 99.72 percent accuracy and a 99.82 percent phishing F1-score. The pretrained iLTM setup reached the absolute highest hard-label score with 99.79 percent accuracy and a 99.86 percent F1-score. Using LIME and feature ablation on the chosen SVM, results showed that predictions relied heavily on security, lexical complexity, and domain structure. When tested on an external source, the source-only XGBoost model dropped to 61.99 percent accuracy and a 72.20 percent phishing F1-score on ISCX-URL2016. Adding 10 percent labeled ISCX data for adaptation raised performance to 91.10 percent accuracy and a 91.28 percent F1-score on the remaining 90 percent. These results prove engineered URL features work great on the native dataset, though accuracy drops when shifting to a different data source.

Keywords—*Phishing URL Detection, Machine Learning, XGBoost, BERT, Tabular Foundation Models, Explainable AI, Cybersecurity*

I. INTRODUCTION

Phishing URLs are a massive cyber threat worldwide, tricking people into giving away passwords, financial details, and other personal data [1]. Globally, billions of malicious phishing emails and links are sent every single day, making it the most common and damaging form of cybercrime. In Bangladesh, the risk is rapidly increasing due to the massive rise in digital financial services, online banking, and mobile transactions. Thousands of cyberattacks target the country's financial and digital sectors every month, with phishing as a primary weapon. Traditionally, people rely on basic security awareness, email spam filters, and manual URL checking to spot these threats. However, these traditional methods often fail [2] because hackers constantly create smart and deceptive links that bypass standard filters. This research is driven by the urgent need for automated detection systems [3]. By using advanced machine learning, contextual models, and data pipelines, this project aims to build strong and reliable tools to protect everyday users from online fraud.

A. Literature Review

Research on phishing detection has explored different data sources, URL features, machine learning models, and evaluation methods. Since these studies use different datasets and experimental settings, their results cannot always be compared directly without considering how each experiment was performed. This section reviews three approaches related to our project: feature-based deep learning, conventional machine learning, and pretrained tabular learning methods [4]–[6].

Barik et al. [4] proposed EGSO-CNN for phishing URL detection using a balanced dataset of phishing and legitimate URLs. The input features were standardized, and a Variational Autoencoder was used to generate a new feature representation. A Convolutional Neural Network was then used for classification, while Enhanced Grid Search Optimization was applied to select the model settings. Their method achieved 99.44% accuracy, 99.21% recall, and a 99.32% F1-score.

Duarte et al. [5] developed a machine learning framework for detecting phishing URLs at an early stage. They used 211,659 URLs collected from sources such as SSL certificate monitoring, domain listings, and reported phishing cases. The data

went through preprocessing, feature engineering, feature selection, and model optimization before different machine learning algorithms were tested. Among them, Light Gradient Boosting Machine (LightGBM) achieved the best performance, with 97.28% accuracy and 96.02% recall. The main goal of the framework was to identify potentially malicious domains early and help prevent phishing attacks.

Bonet et al. [6] introduced the Integrated Large Tabular Model (iLTM) to transfer knowledge across different tabular learning tasks. The model uses tree embeddings, a fixed projection, a hypernetwork-based MLP, and neighboring rows during inference to improve predictions. The authors tested the model on several tabular tasks and with different amounts of labeled data. However, reproducing the complete meta-training process requires significant data and computational resources. Therefore, instead of training iLTM from scratch, our project uses the available pretrained checkpoint and adapts it to the phishing URL dataset.

Although these studies show strong results, their evaluation settings are different, making direct comparison difficult. In particular, random record-level splits may give overly optimistic results when similar or repeated URLs appear in both the training and testing sets. The existing studies also do not provide a single evaluation framework covering conventional classifiers, ensemble methods such as stacking and blending, textual features, contextual models, data augmentation, explainability, and pretrained tabular models. To address these gaps, our project uses a controlled evaluation setup with feature deduplication and a common data split for a fairer comparison of different approaches.

B. Project Scope and Contributions

In this study, phishing URL classification is evaluated using the LegitPhish dataset from Mendeley Data [7]. The experiments use 16 numerical URL features and five classifiers: Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, and XGBoost. We compare baseline and tuned configurations, Stacking and Blending ensembles, training-set augmentation, and textual representations of the numerical features. The project also evaluates BERT, DistilGPT2, and Qwen. It also includes explainability, feature ablation, and three paper-inspired pipelines.

The main contributions of this project are:

- Implementation and evaluation of the five base machine learning classifiers: Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, and XGBoost.
- Comparison of the five base classifiers before and after hyperparameter tuning.
- Construction of Stacking and Blending ensembles using validation-based model ranking.
- Comparison of Random Oversampling, SMOTE, and ADASYN as separate training-data augmentation experiments.
- Implementation of three NLP approaches: TF-IDF, Word2Vec and FastText.
- Evaluation of three contextual approaches: BERT, DistilGPT2, and Qwen2.5-1.5B-Instruct.
- Application of LIME for local model explanation and feature-group ablation for examining the contribution of different URL feature categories.
- Implementation of a WSADBench-inspired labeled phishing ratio experiment.
- Implementation of a StablePFN-inspired feature stability Experiment using bootstrap feature importance and XGBoost.
- Downstream adaptation and label budget evaluation of the official pretrained iLTM model.

The remainder of this report is organized as follows. Section II describes the dataset and preprocessing steps. It also explains the applied models, ensemble methods, augmentation techniques, and paper-inspired pipelines. Section III presents the experimental results and discusses the explainability analysis, ablation study, and comparison with existing research. Section IV concludes the report and discusses possible directions for future research.

II. PROPOSED SYSTEM

A. Dataset

This project uses version 2 of the LegitPhish dataset from Mendeley Data [7]. Each record represents a URL and includes lexical, structural, domain, and security-related information. The original dataset contains 18 columns: the URL, the class label, and 16 numerical features. The URL is kept to identify each record, but it is not used as an input for the conventional classifiers.

The label value 0 represents a phishing URL, whereas 1 represents a legitimate URL. The original file contained 101,219 records. We removed one record with an invalid label and 346 rows that were exact duplicates, which left 100,872 records. Some different URLs still had the same values for all 16 predictors. After keeping one representative from each repeated feature vector, 31,808 records remained. No contradictory labels were found within those repeated feature groups.

We used the feature-deduplicated records to create a common outer split for the experiments. The training pool contains 27,559 records, and the held-out test set contains 4,249 records. The test set contains 3,311 phishing URLs and 938 legitimate URLs. Table I shows how the number of records changed during the different processing steps.

TABLE I: Dataset size at successive processing stages

Processing stage	Records
Original dataset	101,219
Supplied cleaned CSV	100,872
Feature-deduplicated dataset	31,808
Shared outer training pool	27,559
Shared outer test set	4,249

The 16 input features are summarized in Table II. These features describe URL length, path length, character composition, subdomain structure, and security-related indicators. The table also shows the range and average value of each feature.

TABLE II: Dataset features: description, unit, and summary statistics

Feature	Description	Unit	Min	Max	Mean	Type
url_length	Total character count of the URL	Count	7	474	46.41	Discrete
has_ip_address	IP address present in host	Binary	0	1	0.35	Categorical
dot_count	Number of dots in the URL	Count	0	19	3.15	Discrete
https_flag	HTTPS protocol present	Binary	0	1	0.32	Categorical
url_entropy	Shannon entropy of URL string	Bits	2.52	5.87	4.18	Continuous
token_count	Delimited tokens in URL path	Count	1	48	7.65	Discrete
subdomain_count	Subdomain hierarchy levels	Count	-1	5	1.36	Discrete
query_param_count	Key-value query parameters	Count	1	14	1.03	Discrete
tld_length	Character length of the TLD	Count	0	10	3.39	Discrete
path_length	Character count of URL path section	Count	0	317	19.24	Discrete
has_hyphen_in_domain	Hyphen present in domain name	Binary	0	1	0.09	Categorical
number_of_digits	Total digit count (0-9) in URL	Count	0	164	8.89	Discrete
tld_popularity	High-traffic recognized TLD	Binary	0	1	0.32	Categorical
suspicious_file_ext.	High-risk executable file extension	Binary	0	1	0.38	Categorical
domain_name_length	Character length of registered domain	Count	0	36	7.19	Discrete
percentage_numeric_chars	Proportion of numeric characters in URL	Ratio	0.00	65.96	19.46	Continuous

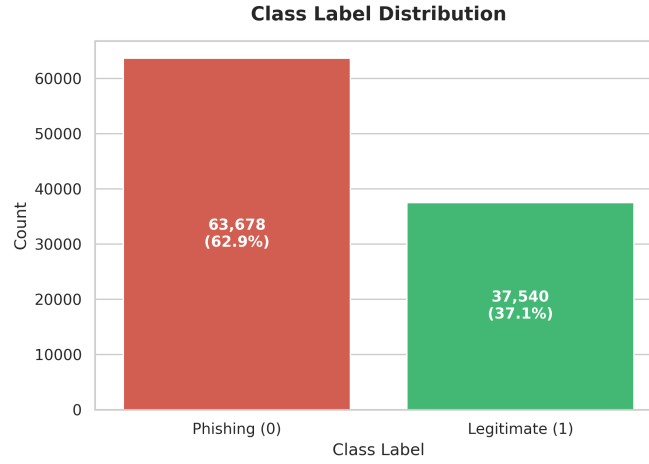


Fig. 1: Class distribution among the valid labeled records in the original dataset, before feature-level deduplication.

Before feature-level deduplication, the valid records included 63,678 phishing URLs and 37,540 legitimate URLs. These values correspond to 62.91% and 37.09% of the data, respectively. Therefore, the original valid records contain noticeably more phishing examples than legitimate examples, as shown in Fig. 1.

We also inspected individual feature distributions before model training. One example is `url_entropy`, which measures the variation of characters within a URL. Fig. 2 compares this feature for phishing and legitimate records and shows how its distribution differs between the two classes.

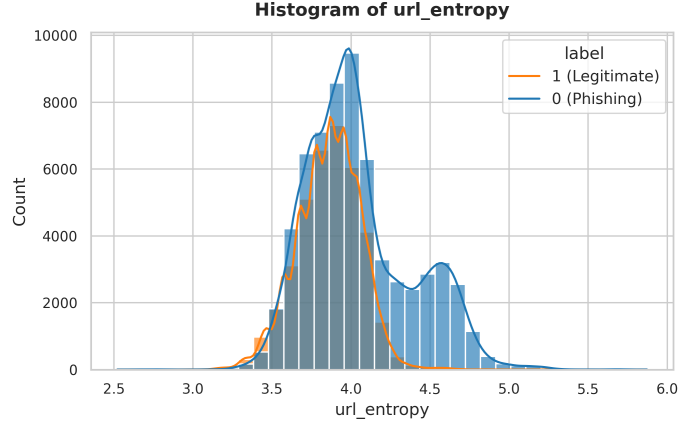


Fig. 2: Distribution of the URL entropy feature for phishing and legitimate URLs in the original dataset.

B. Data Preprocessing

The same cleaned data and outer evaluation split were used across the main experiments. We first checked the expected URL column, the 16 predictor columns, and the target column. Predictor values were converted to numerical form, and the record containing an invalid target value was discarded. In this dataset, class 0 identifies phishing URLs, and class 1 identifies legitimate URLs.

Cleaning was performed at two levels. First, 346 rows that duplicated an entire existing record were removed. This produced the cleaned file of 100,872 records. We then compared records using all 16 predictor values. When several records had the same feature vector and label, one representative was retained. No repeated feature vector had conflicting class labels. This second step reduced the modeling data to 31,808 unique feature records.

To make the evaluation more reliable, structurally similar records were grouped before splitting the data into training and test sets. Continuous features were divided into five quantile bins. These bins were combined with the binary security indicators to create the groups. Twenty percent of the groups were used for testing. This process produced 27,559 training records and 4,249 test records. No row identifiers or structural groups were shared between the two sets. The number of records is not exactly divided into 80% for training and 20% for testing because whole groups were used in each set.

A median imputer was also kept inside the model pipelines to handle any missing numerical values. It was fitted using the outer training partition and then applied to the test partition. The supplied dataset did not contain missing values in its predictor columns, but keeping this step makes the pipeline capable of processing such values. Logistic Regression and SVM used `StandardScaler` to put all features on a similar scale. The scaler was fitted using only the training data. Decision Tree, Random Forest, and XGBoost did not need this scaling and used the numerical features directly.

No additional categorical encoding was required. All predictor columns were already numerical, and the HTTPS, IP-address, domain-hyphen, TLD-popularity, and suspicious-file indicators were represented by zero or one. Applying one-hot or ordinal encoding to these columns would not add new information.

We checked extreme numerical values using the interquartile range (IQR). It was calculated as

$$\text{IQR} = Q_3 - Q_1 \quad (1)$$

where Q_1 and Q_3 are the first and third quartiles. Values below $Q_1 - 1.5 \cdot \text{IQR}$ or above $Q_3 + 1.5 \cdot \text{IQR}$ were considered potential outliers. We found 768 outliers for URL length, 1,475 for dot count, 31 for URL entropy, and 589 for path length. These records were kept because unusually long or complex URLs can be a normal part of phishing behavior.

For the NLP and contextual models, each feature vector was converted into a text sequence containing the feature names and their values. The target label was not included in the input. We also tested Random Oversampling, SMOTE, and ADASYN separately. These methods were applied only to the training data. The test data was kept unchanged.

The structural-group split helps prevent similar feature patterns from appearing in both training and test sets. However, the groups were created using the complete feature-deduplicated data, and the internal validation folds were made at the row level. Therefore, the main experiment is a controlled within-dataset evaluation rather than a fully independent cross-domain test. The separate ISCX-URL2016 experiment provides the external evaluation reported later.

C. Base Machine Learning Models

Five initial machine learning classifiers are implemented.

1) *Logistic Regression*: For binary classification tasks, logistic regression is a probabilistic, linear classifier. It estimates the probability that a URL belongs to a particular class. The logistic function is:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (2)$$

L1, L2, and Elastic-Net regularization are used to reduce overfitting.

2) *SVM, or support vector machine*: SVM aims to find the best hyperplane that maximally separates the classes. To deal with non-linear separability, it employs kernel tricks. The definition of the decision function is:

$$f(x) = \text{sign}(w^T \phi(x) + b) \quad (3)$$

Where:

- w is the weight vector,
- $\phi(x)$ maps input to a higher-dimensional space,
- b is the bias term.

3) *Decision Tree*: Decision Tree classifiers use a tree-like structure to describe decision paths based on feature values. To maximize information gain, the nodes divide the dataset into subgroups based on factors like entropy or Gini impurity:

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2 \quad (4)$$

Where:

- S is the dataset or subset being evaluated,
- C is the total number of classes,
- p_i is the proportion (probability) of samples belonging to class i in set S .

Overfitting is controlled through:

- Maximum depth.
- Minimum samples per leaf.
- Minimum samples for splitting.
- Cost-complexity pruning.

4) *Random Forest*: [8] is an ensemble learning method that uses multiple decision trees. It combines their predictions using majority voting. . Doing this prevents the model from memorizing the data and helps it handle unseen links more effectively. The final forecast, $\hat{y}$ is provided by:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_k(x)\} \quad (5)$$

Where:

- $T_i(x)$ is the prediction from the i^{th} tree.

The final output is produced using majority voting.

5) *XGBoost*: XGBoost [9] (Extreme Gradient Boosting) is an optimized distributed gradient boosting library. This is designed to be highly efficient, flexible, and portable. It builds an ensemble of decision trees sequentially, where each new tree aims to minimize a regularized objective function combining the training loss and model complexity. The objective function at step t is given by:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (6)$$

Where:

- $l(y_i, \hat{y}_i)$ is a differentiable convex loss function measuring the difference between the prediction $\hat{y}_i$ and the target y_i .
- $f_t(x_i)$ represents the prediction of the t -th decision tree for instance x_i .
- $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ is the regularization term penalizing model complexity,
- T is the number of leaves in the tree f_t and w represents leaf weights.

D. Hyperparameter Optimization

The five conventional classifiers were tuned using `RandomizedSearchCV`. The search objective was phishing recall, with phishing represented by class 0. All searches used a random seed of 42.

Logistic Regression, Decision Tree, Random Forest, and XGBoost used five-fold cross-validation. SVM used a three-fold cross-validation. The number of sampled candidate configurations was 20, 25, 24, 25, and 16, respectively.

Table III lists the actual candidate values and selected settings recorded by the executed notebook. These are discrete candidate lists; The randomized search samples combinations from them.

TABLE III: Hyperparameter Search Spaces and Selected Settings

Model	Hyperparameter	Candidate values	Selected
Logistic Regression	C	0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1	0.5
	Penalty	L1, L2, Elastic Net	Elastic Net
	$l1_ratio$	0.1, 0.3, 0.5, 0.7, 0.9	0.3
	$class_weight$	None, balanced	None
SVM	C	0.05, 0.1, 0.25, 0.5, 1, 2	0.25
	Kernel	rbf, poly	poly
	γ	scale, auto, 0.001, 0.005, 0.01, 0.05	auto
	$class_weight$	None, balanced	None
Decision Tree	max_depth	3, 4, 5, 6, 7, 8, 10, 12	12
	$min_samples_split$	10, 20, 30, 50	20
	$min_samples_leaf$	5, 10, 15, 20	15
	ccp_alpha	0, 0.0001, 0.0005, 0.001, 0.005	0.005
	Criterion	gini, entropy	gini
	$class_weight$	None, balanced	None
Random Forest	$n_estimators$	150, 200, 250, 300, 400	150
	max_depth	5, 7, 9, 11, 13	5
	$min_samples_split$	10, 20, 30	30
	$min_samples_leaf$	3, 5, 8, 12, 15	12
	$max_features$	sqrt, log2	sqrt
	ccp_alpha	0, 0.0001, 0.0005, 0.001	0.0005
	$class_weight$	None, balanced, balanced_subsample	None
XGBoost	$n_estimators$	100, 150, 200, 250, 300, 400	150
	max_depth	2, 3, 4, 5	3
	$learning_rate$	0.01, 0.02, 0.03, 0.05, 0.08	0.02
	min_child_weight	3, 5, 8, 10	3
	$subsample$	0.6, 0.7, 0.8, 0.9	0.6
	$colsample_bytree$	0.5, 0.6, 0.7, 0.8	0.8
	γ	0, 0.05, 0.1, 0.2	0.2
	reg_alpha	0.05, 0.1, 0.2, 0.5, 1	1
	reg_lambda	1, 2, 3, 5, 8	1

Logistic Regression, Decision Tree, Random Forest, and XGBoost was searched using the complete outer training pool. To reduce computation, the SVM search used a reproducible random subset of at most 25,000 outer-training records. This subset was sampled without explicit stratification. After selection, the SVM configuration was refitted on the complete outer training pool.

After the XGBoost search, the selected configuration was trained using an internal stratified split that reserved 15% of the outer training pool for validation. Early stopping used a patience of 20 boosting rounds. Consequently, the selected estimator count represents the configured upper limit rather than necessarily the number of boosting rounds retained after early stopping.

The outer test set was not used to score search candidates, select hyperparameters, or control XGBoost early stopping.

E. Overfitting Control

Different forms of regularization and validation are used according to each model's requirements.

- **Logistic Regression:** The search includes L1, L2, and Elastic Net penalties. The selected model uses Elastic-Net regularization with $C = 0.5$ and an L1 mixing ratio of 0.3.

- **SVM:** Margin regularization is controlled using the parameter C . The selected SVM uses $C = 0.25$, which applies stronger regularization than larger values of C .
- **Decision Tree:** Maximum depth, minimum split size, minimum leaf size, and Cost-complexity pruning is used to limit the excessive tree growth.
- **Random Forest:** Maximum tree depth, minimum split and leaf sizes, feature Subsampling and cost-complexity pruning restrict the complexity of the individual trees.
- **XGBoost:** Learning-rate reduction, row and column subsampling, minimum child weight, split penalties, L1/L2 regularization Early stopping is used to control model complexity.
- **Stacking:** Three-fold out-of-fold predictions are used to create the meta-model training features. This prevents the meta-model from being trained directly on in-sample predictions from base models fitted on the same records.
- **Contextual Models:** BERT and DistilGPT2 use validation-based checkpoint selection and early stopping. Their outer-test results are calculated only after the configuration has been selected.

For the machine learning models, the notebook defines an accuracy generalization gap as:

$$G_{\text{Acc}} = \text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{test}}. \quad (7)$$

It also calculates a balanced-accuracy gap:

$$G_{\text{BA}} = \text{BA}_{\text{train}} - \text{BA}_{\text{test}}. \quad (8)$$

A large positive gap can indicate that a model fits the training data more strongly than the held-out data. A small gap alone does not prove good generalization because an underfitted model can perform poorly on both partitions. Therefore, generalization gaps must be interpreted together with the held-out accuracy, balanced accuracy, phishing recall, and phishing F1.

The outer test set is used to characterize final model performance. It is not used as the scoring data within Randomized-SearchCV.

F. Validation Ranking for Ensemble Design

After hyperparameter tuning, the five tuned base models are ranked to assign their roles in the required ensemble design. A stratified 20% internal validation partition is created from the outer training pool using random seed 2026.

Models are ranked first by validation balanced accuracy, by phishing F1, and by phishing recall. The outer test set is not used for assigning the ensemble roles.

TABLE IV: Validation ranking used to assign ensemble roles. All metric values are percentages.

Rank	Model	Bal. Acc.	F1	Recall	Assigned role
1	SVM	98.97	99.52	99.49	Meta-model
2	Logistic Regression	98.71	99.26	98.97	Base learner
3	Decision Tree	97.01	99.14	99.98	Base learner
4	Random Forest	97.01	99.14	99.98	Base learner
5	XGBoost	97.01	99.14	99.98	Not selected

The ranking selects SVM as the meta-model. Logistic Regression, Decision Tree and Random Forest become the three base learners. Decision Tree, Random Forest, and XGBoost have identical reported ranking metrics, so their recorded order is retained when assigning the required ranks.

The ranking partition is separate from the outer test set. It is drawn from the same outer training pool previously used for hyperparameter search. Therefore, it is used as an internal architecture-ranking partition rather than being described as a completely untouched validation dataset.

G. Stacking

The Stacking ensemble uses the three selected base learners: Logistic Regression, Decision Tree, and Random Forest. The final estimator is the selected SVM meta-model.

For each training record, the base learners generate out-of-fold probability features using a three-fold cross-validation. These probability outputs become the input features for the SVM meta-model. The original 16 URL features are not passed directly to the meta-model.

The stacking prediction can be expressed as:

$$\hat{y} = H(p_1(x), p_2(x), p_3(x)), \quad (9)$$

where $p_1(x)$, $p_2(x)$ and $p_3(x)$ are the probability outputs of Logistic Regression, Decision Tree, and Random Forest, respectively, and H is the SVM meta-model.

After the out-of-fold meta-training procedure, the complete stacking classifier is fitted using the outer training pool. Its final performance is evaluated once on the shared outer test set.

H. Blending

Blending uses the same three base learners and SVM meta-model as Stacking, but it creates the meta-model training data using a fixed holdout rather than cross-validation.

A stratified 20% blending holdout is created from the outer training pool using random seed 2027. Logistic Regression, Decision Tree and Random Forest are fitted on the remaining 80% of the outer training pool. They generate probability outputs for the blending holdout.

The blending feature vector can be written as:

$$z(x) = [p_{LR}(x), p_{DT}(x), p_{RF}(x)]. \quad (10)$$

The SVM meta-model is trained using these three probability features and the corresponding labels of the blending holdout. For outer-test prediction, the three fitted base learners first produce the same probability features, which are passed to the fitted SVM meta-model.

Stacking and blending are evaluated as separate experiments. The primary six-model table contains the five tuned base models and Stacking. Blending is reported separately as the additional mandatory ensemble experiment.

I. Data Augmentation

The outer training pool contains 21,438 phishing records and 6,121 legitimate records. Therefore, legitimate URLs form the minority training class in this processed dataset.

Three augmentation and resampling methods are evaluated as separate experiments. In every experiment, resampling is applied only to the outer training pool. The shared outer test set is never resampled.

The tuned SVM selected by the internal validation ranking is used as the reference classifier for all three augmentation methods. Using the same classifier allows for the effect of the resampling method to be compared without changing the classification algorithm.

- 1) **Random Oversampling:** Random Oversampling duplicates existing legitimate training records until the two training classes contain equal numbers of records. After resampling, the training data contains 21,438 phishing and 21,438 legitimate records.
- 2) **Synthetic Minority Over-sampling Technique:** SMOTE [10] generates new minority-class samples by interpolating between a legitimate record and one of its nearest legitimate neighbors. The experiment uses five nearest neighbors and a random seed of 42.

The interpolation can be expressed as:

$$x_{\text{new}} = x_i + \lambda(x_j - x_i) \quad (11)$$

where x_i is a legitimate training instance, x_j is one of its selected nearest legitimate neighbors, and $\lambda \in [0, 1]$ is a random interpolation value. SMOTE produces 21,438 records in each training class.

- 3) **Adaptive Synthetic Sampling:** ADASYN [11] gives greater sampling weight to minority records located in more difficult classification regions. It also uses five nearest neighbors and a random seed of 42. The resulting training data contains 21,438 phishing and 21,457 legitimate records.

For each augmentation method, a new copy of the selected SVM pipeline is fitted on the corresponding resampled training data. The pipeline applies feature standardization after resampling. The fitted classifier is evaluated on the unchanged shared test set of 4,249 records.

The augmentation experiments are compared using accuracy, balanced accuracy, phishing precision, phishing recall, phishing F1, and ROC-AUC. These experiments measure whether Resampling changes performance; they do not assume that augmentation must improve the classifier.

Random Oversampling preserves existing feature values, whereas SMOTE and ADASYN interpolate in the numerical feature space. Consequently, the synthetic methods can produce intermediate values for features that originally behave like counts or binary indicators. This is taken into account when interpreting their results.

J. NLP Models

Our three NLP experiments turn the same 16 URL features into simple text. The URL ID and target label are not used. Each record contains the feature names and their values, for example:

```
url_length: 65 | dot_count: 3 | https_flag: 1 | url_entropy: 4.2 | ...
```

These experiments check how NLP models work with feature-based text. They do not use webpage content.

The 27,559 training records were split into 23,425 training records and 4,134 validation records. We used a stratified 15% split with random seed 42. The NLP models were trained using the 23,425 training records. Candidate configurations are ranked using validation balanced accuracy, followed by phishing F1 and phishing recall.

After selecting a configuration, the final NLP pipeline is refitted using both the inner training and validation records. It is evaluated on the shared outer test set of 4,249 records.

1) *TF-IDF*: TF-IDF [12] represents the serialized feature-value text by weighting terms according to their frequency within one record and their rarity across the complete training corpus. It can be expressed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (12)$$

where t is a term and d is a serialized record. A Logistic Regression classifier is fitted using the resulting sparse TF-IDF vectors.

The fixed baseline configuration uses unigrams, a maximum of 10,000 features, a minimum document frequency of 2, no sublinear term-frequency scaling, and Logistic Regression with $C = 1$.

The validation search evaluates unigram and unigram-bigram representations, maximum vocabulary sizes of 5,000, 10,000, and 20,000, optional sublinear term-frequency scaling, and several Logistic Regression regularization strengths.

The selected configuration uses unigrams and bigrams, a maximum of 5,000 TF-IDF features, a minimum document frequency of 2, no sublinear term-frequency scaling, and Logistic Regression with $C = 5$.

2) *Word2Vec*: Word2Vec maps the tokens in the serialized feature-value text to dense vectors [13]. Before training, the text is converted to lowercase, and the colon and vertical-bar separators are removed. The remaining strings are split into tokens.

The vector representation of a complete record is calculated by averaging its available token vectors:

$$v_d = \frac{1}{|T_d|} \sum_{t \in T_d} v_t, \quad (13)$$

where T_d is the set of recognized tokens in record d and v_t is the learned vector of token t . Logistic Regression classifies the averaged record vectors.

The baseline uses 100-dimensional embeddings, context window 5, CBOW training, minimum token count 1, and Logistic Regression with $C = 1$. The validation search compares embedding dimensions of 50, 100, and 200; context windows of 3 and 5; CBOW and Skip-Gram training, and several Logistic Regression models' regularization strengths.

The validation-selected Word2Vec configuration uses 100-dimensional embeddings, a context window of 5, CBOW, a minimum token count of 1, and Logistic Regression with $C = 1$. These are the same settings used by the baseline. The final pipeline differs because it is refitted using the combined inner training and validation data.

3) *FastText*: FastText extends word-vector learning by constructing a token representation from character-level subwords [14]. This representation can preserve information from numerical and feature-name tokens that share character patterns.

As in the Word2Vec experiment, token embeddings belonging to One serialized record is averaged before the logistic regression classification.

The baseline uses 100-dimensional vectors, context window 5, CBOW, minimum token count 1, character subwords of length 3 to 6, and Logistic Regression with $C = 1$.

The validation search compares vector dimensions of 50 and 100, context windows of 3 and 5, CBOW and Skip-Gram, subword ranges of 2-5 and 3-6 characters, and several Logistic Regression regularization strengths.

The validation-selected FastText configuration uses 100-dimensional vectors, a context window of 5, CBOW, and a minimum token count of 1, character subwords of length 3 to 6, and Logistic Regression with $C = 1$. As with Word2Vec, these selected settings match the baseline settings, while the final model is refitted using more records.

4) *NLP Comparison Scope*: The terms "baseline" and "selected" identify the initial inner-training fit and the validation-selected/refitted pipeline, respectively. For Word2Vec and FastText, the selected hyperparameters match their baseline settings. Consequently, changes in their final test results cannot be attributed solely to different hyperparameters; the final refit also uses the additional validation records.

The embedding models use four worker threads and a random seed of 42. Parallel embedding training may still introduce small run-to-run numerical variation. ROC-AUC values were not exported for these NLP experiments, so their comparison uses accuracy, balanced accuracy, phishing precision, phishing recall, phishing F1, and MCC.

K. Contextual and LLM Models

Three contextual model approaches are evaluated using the same 16 numerical URL features and a shared outer test partition as the other experiments. The numerical feature names and values are serialized into text. The URL identifier and target label are excluded from the model input.

1) *Corrected Evaluation Protocol*: Verification of the earlier shared implementation identified test-set use during final transformer checkpoint selection. The BERT and DistilGPT2 experiments were therefore revised so that checkpoint selection and early stopping use only inner validation data. The selected checkpoint is subsequently evaluated on the shared outer test set.

The earlier GPT-family implementation also used a 128-token input limit that could truncate the serialized feature records. The revised experiment uses a maximum length of 256 tokens for BERT and DistilGPT2, together with input-length checks to confirm that the 16 serialized features are retained.

The corrected contextual experiments are executed in a focused companion notebook. This notebook preserves the original dataset cleaning, feature order, outer training pool, and 4,249-record test partition. The focused notebook repairs the affected contextual experiments; it does not create a new dataset or independent project.

The final contextual results reported in this paper come from the combined executed run containing BERT, DistilGPT2, and Qwen. The earlier contextual results are not mixed with this run.

2) *BERT*: We use the pretrained `bert-base-uncased` model for binary sequence classification [15]. BERT processes the serialized feature-value record and produces probabilities for the phishing and legitimate classes.

A balanced subset of 2,500 inner-training records is used for model fitting. A separate balanced subset of 500 records from the inner validation partition is used for checkpoint and configuration selection.

The fixed baseline configuration uses:

- learning rate 2×10^{-5} ;
- maximum two epochs;
- batch size 16;
- weight decay 0.01; and
- maximum input length 256 tokens.

Three predefined candidate configurations are evaluated. Candidates are ranked using validation balanced accuracy, followed by phishing F1 and phishing recall.

The selected BERT candidate uses:

- learning rate 3×10^{-5} ;
- maximum four epochs;
- batch size 16;
- weight decay 0.05; and
- early stopping based on validation performance.

The selected candidate stopped after three training epochs. Final evaluation uses the saved checkpoint with the best validation result rather than the last checkpoint automatically.

3) *DistilGPT2*: We use `distilgpt2` as the GPT-family sequence classifier [16]. This is a locally executed pretrained GPT-family model. It is not a ChatGPT or OpenAI API experiment.

A balanced subset of 2,200 inner-training records is used for fitting, and a separate balanced validation subset of 450 Records are used for configuration and checkpoint selection.

The fixed baseline configuration uses:

- learning rate 5×10^{-5} ;
- one training epoch;
- batch size 16;
- weight decay 0.01; and
- maximum input length 256 tokens.

Three predefined candidate configurations are compared using validation balanced accuracy, followed by phishing F1 and phishing recall.

The selected DistilGPT2 candidate uses:

- learning rate 2×10^{-5} ;
- maximum three epochs;
- batch size 16;
- weight decay 0.01; and
- validation-based checkpoint selection.

The selected configuration trains for three epochs. The best validation checkpoint is evaluated on the outer test set.

TABLE V: Contextual-model experiment configurations

Model	Training records	Validation records	Baseline	Selected adaptation
BERT	2,500 balanced	500 balanced	Two-epoch weight training	Validation-selected weight fine-tuning; learning rate 3×10^{-5} ; batch 16; weight decay 0.05
DistilGPT2	2,200 balanced	450 balanced	One-epoch weight training	Validation-selected weight fine-tuning; learning rate 2×10^{-5} ; batch 16; weight decay 0.01
Qwen2.5-1.5B-Instruct	Frozen weights	500 balanced	Zero-shot prompt	Validation-selected two-example prompt; one example from each class.

4) *Qwen2.5-1.5B-Instruct*: Qwen2.5-1.5B-Instruct is evaluated as an open-weight instruction model [17]. Qwen receives prompts containing the same serialized 16 numerical URL features.

Unlike BERT and DistilGPT2, Qwen’s pretrained model weights remain frozen. The experiment performs a discrete prompt example selection rather than gradient-based weight fine-tuning.

Three prompt configurations are compared:

- zero-shot, with no labeled examples;
- two-shot, with one phishing and one legitimate inner-training example; and
- four-shot, with two examples from each class.

The example records are drawn only from inner training. The three configurations are compared using a balanced validation sample of 500 records. Their validation balanced accuracies are 50.00%, 94.00%, and 84.80% for zero, two and four examples, respectively. Therefore, the two-example prompt is selected before the outer-test evaluation.

The model is instructed to represent phishing as option A and legitimate as option B. The next-token logits of A and B are normalized over those two options to form a phishing ranking score. Classification uses a fixed threshold of 0.5.

These A/B scores are not assumed to be calibrated probabilities. The outer test labels are not included in the prompts and are not used for prompt selection.

5) *Contextual Experiment Summary*: Table V summarizes the different adaptation procedures. The experiments share the same outer test set, but their fitting and adaptation budgets are not identical.

L. Paper-Inspired Pipelines

Three paper-based pipeline contributions are implemented: a WSADBench-inspired phishing-label budget experiment, a StablePFN-inspired feature-stability adaptation, and downstream application of the pretrained iLTM model. Each implementation uses the same shared outer test set, but the three methods evaluate different research questions.

1) *WSADBench-Inspired Phishing-Label Budget Experiment*: The first paper-based pipeline adapts the Ratio of Labeled Anomalies idea from WSADBench [18]. The objective is to measure how a classifier behaves when only a limited fraction of the phishing labels is available during training.

This project applies the label-ratio concept to supervised phishing classification. It does not reproduce the complete WSAD-Bench benchmark, its complete collection of algorithms, or its experiments across multiple data modalities.

The shared outer training pool contains 21,438 phishing and 6,121 legitimate records. All 6,121 legitimate labels remain available at every ratio. The available phishing-label ratios are:

$$\gamma \in \{1\%, 5\%, 10\%, 25\%, 50\%, 100\%\}. \quad (14)$$

For each value of γ , a reproducible subset of the phishing training records is retained. Phishing records not selected for that ratio are excluded from the corresponding training pool; they are not used as unlabeled examples.

The resulting phishing-label counts are 214, 1,071, 2,143, 5,359, 10,719, and 21,438. A separate Random Forest is fitted for every ratio using the following fixed configuration:

- 300 trees;
- maximum depth 10;
- minimum leaf size 5;
- square-root feature sampling;
- balanced class weighting; and
- random seed 42.

Every fitted model is evaluated on the same untouched outer test set of 4,249 records. This makes it possible to compare performance across the available phishing-label ratios without changing the test population.

2) *StablePFN-Inspired Feature-Stability Adaptation*: The second paper-based pipeline is inspired by the stable prediction objective discussed in StablePFN [19]. The implemented method is a lightweight feature-stability adaptation. It does not execute or reproduce the pretrained causal-aware StablePFN foundation model.

Five bootstrap samples are drawn from the complete outer training pool. A Random Forest containing 150 trees is fitted on each sample. Every tree ensemble uses a maximum depth of 8, a minimum leaf size of 5, and square-root feature sampling.

For feature j , the stability score is calculated as:

$$S_j = \frac{\bar{I}_j}{\sigma(I_j) + 10^{-6}}, \quad (15)$$

where $\bar{I}_j$ is the mean Random Forest importance of feature j across the five bootstrap fits, and $\sigma(I_j)$ is its importance standard deviation.

A high score indicates that a feature has relatively high average importance compared with its variability across the bootstrap samples. The ten highest-ranked features are retained:

- path length;
- number of digits;
- URL length;
- dot count;
- percentage of numeric characters;
- token count;
- HTTPS flag;
- suspicious file-extension indicator;
- TLD length; and
- subdomain count.

A regularized XGBoost classifier is fitted using these ten selected features. It uses 200 estimators, maximum depth 3, learning rate 0.03, minimum child weight 5, row subsampling 0.8, column subsampling 0.8, L1 regularization 0.2, L2 regularization 3, and random seed 42.

This experiment evaluates the implemented bootstrap feature-stability procedure. Its results must be reported as “StablePFN-inspired” rather than as results produced by the original StablePFN model.

3) *iLTM: Integrated Large Tabular Model*: The third paper-based contribution applies the official pretrained iLTM implementation of Bonet et al. [6] to phishing URL classification. This experiment performs the downstream adaptation of a released iLTM checkpoint. It does not repeat the original multi-dataset experiment. The meta-training procedure was used to create the pretrained model.

The experiment uses the authors’ pretrained `xgbrconcat` checkpoint. This checkpoint combines XGBoost-derived leaf embeddings with robustly preprocessed numerical features. It must not be described as simultaneously combining XGBoost and CatBoost leaf embeddings.

The processed representation includes a random-feature expansion of dimension 32,768, followed by PCA projection to 512 dimensions. The pretrained hypernetwork uses the training-set representation to generate the parameters of a three-layer MLP predictor. The generated predictor is subsequently fine-tuned using the available labeled records.

The iLTM experiment preserves the shared 31,808-record feature-deduplicated dataset, 27,559-record outer training pool, and 4,249-record outer test set. Phishing remains class 0, and legitimate remains class 1.

The same six phishing-label ratios used in the RF experiment are evaluated. All 6,121 legitimate training-pool labels are retained, while 1%, 5%, 10%, 25%, 50%, or 100% of the 21,438 phishing labels are available. The remaining phishing records were excluded from the training data.

Within each available label pool, iLTM reserves 15% for validation-based fine-tuning control. The reported total label budget includes both the fitting and validation records. Table VI presents the exact accounting.

The main iLTM settings are:

- one generated predictor;
- generation batch size 512;
- maximum 30 fine-tuning epochs;
- fine-tuning batch size 128;
- prediction batch size 128;
- validation-based early stopping;
- cosine-similarity retrieval;
- retrieval weight $\alpha = 0.30$;
- retrieval temperature $\tau = 1.0$; and
- maximum retrieval context of 4,096 records.

TABLE VI: iLTM label-budget accounting

Phishing (%)	Phishing	Legitimate	Total labels	iLTM fit	iLTM validation
1	214	6,121	6,335	5,384	951
5	1,071	6,121	7,192	6,113	1,079
10	2,143	6,121	8,264	7,024	1,240
25	5,359	6,121	11,480	9,758	1,722
50	10,719	6,121	16,840	14,314	2,526
100	21,438	6,121	27,559	23,425	4,134

The matched RF reference uses the same total available label pool and the same outer test set at every ratio. RF is fitted on the complete available pool, while iLTM reserves its 15% internal validation subset. Therefore, the comparison uses the same label budget but not the same number of optimizer-fitting records.

4) *iLTM Retrieval Ablation*: To examine the contribution of retrieval during prediction, a second set of predictions is generated with retrieval disabled. The ablation uses the same fitted iLTM weights as the corresponding main experiment at each label ratio. It is not a separately trained retrieval-free model.

Consequently, this ablation evaluates the immediate effect of turning off retrieval during inference. It does not isolate the effect that retrieval may have had during fine-tuning or during the original pretraining of iLTM.

All iLTM, matched RF, and retrieval-ablation results are evaluated using accuracy, balanced accuracy, phishing precision, phishing recall, phishing F1, Matthews correlation coefficient, AUROC, and Average Precision.

M. Evaluation Metrics

1) *Positive-Class Convention*: The original dataset represents phishing URLs with class 0 and legitimate URLs with class 1. For evaluation, phishing is treated as the positive class. Therefore, TP denotes a phishing URL correctly classified as phishing, while FN denotes a phishing URL incorrectly classified as legitimate. FP represents a legitimate URL classified as phishing, and TN represents a legitimate URL correctly classified as legitimate.

2) *Classification Metrics*: Accuracy measures the proportion of all test samples that are classified correctly:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (16)$$

Phishing precision measures how many URLs predicted as phishing are actually phishing:

$$Precision = \frac{TP}{TP + FP}. \quad (17)$$

Phishing recall measures the proportion of phishing URLs detected by the model:

$$Recall = \frac{TP}{TP + FN}. \quad (18)$$

The F1-score is the harmonic mean of phishing precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}. \quad (19)$$

Specificity measures the proportion of legitimate URLs that are correctly recognized:

$$Specificity = \frac{TN}{TN + FP}. \quad (20)$$

Balanced accuracy gives equal importance to phishing recall and legitimate specificity:

$$Balanced\ Accuracy = \frac{Recall + Specificity}{2}. \quad (21)$$

3) *Threshold-Independent Metrics*: ROC-AUC shows how well the model separates phishing URLs from legitimate ones. AP (AUPRC) summarizes precision and recall. MCC gives a balanced measure, especially for imbalanced data.

All metrics are calculated on the same test set. We mainly focus on phishing recall and F1-score because missing a phishing URL is more serious than a false alarm.

N. Overall System Workflow

Dataset validation, deduplication, and structural grouping are completed before the outer split. Model development uses only the outer training pool, while the held-out test set remains isolated until the final evaluation.

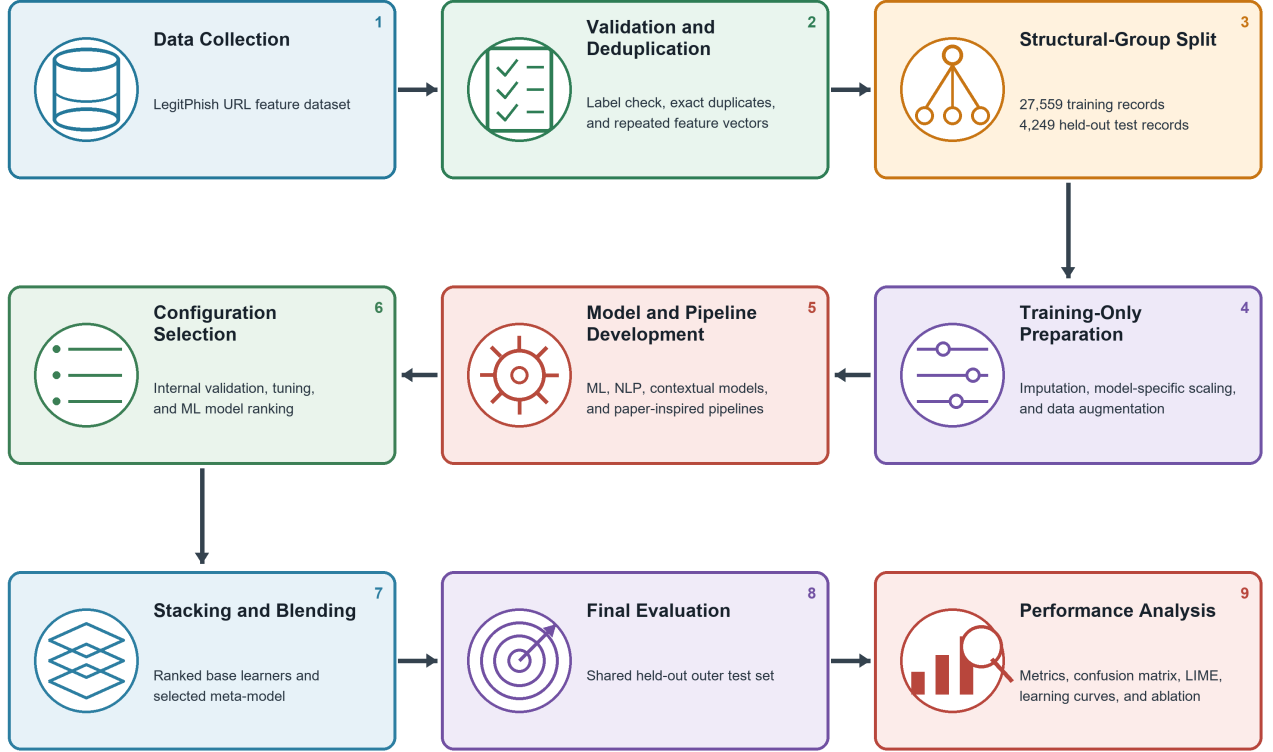


Fig. 3: Complete workflow of the proposed phishing URL detection and evaluation framework. The training branch covers preprocessing, augmentation, model development, configuration selection, and ensemble construction. The held-out outer test set is used only for the final evaluation.

The training branch includes missing-value handling, model-specific feature scaling, internal validation, and training-set augmentation. Five conventional classifiers are first tuned and ranked using internal validation results. The highest-ranked classifier becomes the meta-model, while the classifiers ranked second, third, and fourth become the base learners for Stacking and Blending.

The NLP, contextual-model, and paper-inspired pipelines are developed using their respective training or adaptation procedures. After model selection and fitting are complete, all reported methods are evaluated using the same held-out outer test set. The final stage covers metric comparison, the normalized confusion matrix, learning-curve analysis, LIME explainability, and feature or retrieval ablation.

III. RESULTS AND DISCUSSION

All models were evaluated on the common outer test set containing 4,249 samples, including 3,311 phishing URLs and 938 legitimate URLs. Phishing is treated as the positive class. Percentage-based results are rounded to two decimal places, while ROC-AUC is reported on a scale from 0 to 1.

A. Baseline Machine Learning Results

Table VII presents the performance of the five untuned machine learning models.

B. Primary Six-Model Comparison

Following hyperparameter selection, the primary comparison contains the five required machine learning models and the stacking ensemble. The results are presented in Table VIII.

TABLE VII: Baseline Machine Learning Results

Model	Acc. (%)	Precision (%)	Recall (%)	F1 (%)
Logistic Regression	97.79	99.57	97.58	98.57
SVM	98.59	99.72	98.46	99.09
Decision Tree	99.55	99.76	99.67	99.71
Random Forest	99.62	99.52	100.00	99.76
XGBoost	99.72	99.70	99.94	99.82

TABLE VIII: Primary Six-Model Results After Tuning

Model	Acc. (%)	BA (%)	Precision (%)	Recall (%)	F1 (%)
Logistic Regression	98.00	98.22	99.60	97.83	98.70
SVM	98.59	98.71	99.69	98.49	99.09
Decision Tree	97.79	94.99	97.24	100.00	98.60
Random Forest	97.79	94.99	97.24	100.00	98.60
XGBoost	97.79	94.99	97.24	100.00	98.60
Stacking	97.79	94.99	97.24	100.00	98.60

TABLE IX: Stacking and Blending Comparison

Model	Acc.	BA	Precision	Recall	F1	AUC
Stacking	97.79	94.99	97.24	100.00	98.60	0.9529
Blending	97.95	97.08	98.73	98.64	98.69	0.9985

C. Additional Blending Experiment

Blending was evaluated as an additional mandatory ensemble experiment. Stacking is repeated in Table IX only to provide a direct comparison between the two ensemble designs.

The accuracy, balanced accuracy, precision, recall, and F1 entries in Table IX are percentages.

D. Data Augmentation Results

The tuned SVM configuration was used to compare the three training sets augmentation strategies.

TABLE X: SVM Performance After Training-Set Augmentation

Method	Accuracy (%)	BA (%)	Precision (%)	Recall (%)	F1 (%)	AUC
Random Oversampling	97.93	98.56	99.91	97.43	98.65	0.9968
SMOTE	97.95	98.53	99.88	97.49	98.67	0.9966
ADASYN	97.51	98.28	99.91	96.89	98.37	0.9959

For comparison, the tuned SVM without augmentation obtained 98.59% accuracy, 98.71% balanced accuracy, 99.69% phishing precision, 98.49% phishing recall, 99.09% phishing F1, and 0.9970 ROC-AUC.

E. NLP Baseline and Selected Results

Table XI compares the initial and validation-selected configurations of the three NLP representations.

TABLE XI: Baseline and Selected NLP Model Performance

Model	Configuration	Acc. (%)	BA (%)	Prec. (%)	Recall (%)	F1 (%)
TF-IDF + LR	Baseline	89.46	89.64	96.92	89.31	92.96
TF-IDF + LR	Selected	94.54	93.67	97.71	95.23	96.45
Word2Vec + LR	Baseline	90.47	92.39	98.69	88.95	93.57
Word2Vec + LR	Selected	91.43	92.82	98.55	90.34	94.26
FastText + LR	Baseline	94.96	94.82	98.41	95.08	96.71
FastText + LR	Selected	95.27	94.94	98.35	95.53	96.92

All percentage-based entries in Table XI uses phishing as the positive class. The selected configurations were chosen using the internal validation partition before evaluation on the shared outer test set. ROC-AUC is not reported because the executed experiment did not export probability-based NLP predictions.

F. Contextual and LLM Results

The three contextual experiments consist of BERT, DistilGPT-2, and Qwen2.5-1.5B-Instruct. BERT and DistilGPT-2 use validation-selected weight-training configurations, whereas Qwen uses frozen model weights with a validation-selected prompt.

TABLE XII: Baseline and Selected Contextual-Model Performance

Model	Configuration	Acc. (%)	BA (%)	Prec. (%)	Recall (%)	F1 (%)
BERT	Baseline	97.03	95.42	97.89	98.31	98.10
BERT	Selected	97.81	95.84	97.86	99.37	98.61
DistilGPT-2	Baseline	90.73	91.07	97.46	90.46	93.83
DistilGPT-2	Selected	92.23	94.18	99.27	90.70	94.79
Qwen2.5-1.5B	Zero-shot	77.92	50.00	77.92	100.00	87.59
Qwen2.5-1.5B	Two-shot prompt	89.79	91.15	98.00	88.70	93.12

All percentage-based entries in Table XII uses phishing as the positive class. BERT and DistilGPT-2 use validation-selected weight-training configurations. Qwen uses frozen model weights; its selected configuration changes only the prompt from zero-shot to a balanced two-shot prompt. Therefore, the Qwen experiment should be described as prompt selection rather than model fine-tuning.

G. NLP and Contextual Model Efficiency

Table XIII reports the resource measurements for the selected NLP and contextual configurations. The NLP measurements were reproduced using the same cleaned training pool in the supplementary diagnostic notebook. BERT and DistilGPT-2 values were obtained from the training receipts saved by the corrected contextual-model experiment.

TABLE XIII: Selected NLP and Contextual-Model Resource Measurements

Model	Parameters	Size (MiB)	Epochs	Training (s)	Inference (s)	Hardware
TF-IDF + LR	10,001	0.076	–	1.255	–	Colab CPU
Word2Vec + LR	2,458,501	9.379	5	8.731	–	Colab CPU
FastText + LR	202,458,501	772.318	5	22.638	–	Colab CPU
BERT	109,483,778	417.647	3	79.450	–	Tesla T4
DistilGPT-2	81,914,112	312.478	3	62.851	–	Tesla T4
Qwen2.5-1.5B	1,543,714,304	2,944.401	0	–	389.958	Tesla T4

For TF-IDF, Word2Vec, and FastText, training time includes both representation, construction, and Logistic Regression fitting. Approximate size represents the stored learned numerical arrays and does not include Python-object overhead. BERT and DistilGPT-2 sizes represent approximate full-precision checkpoint weights. Qwen uses frozen half-precision weights; therefore, it has no training epochs or training time, and the table reports the full-test inference time of its selected two-shot prompt. Because the NLP and contextual measurements were collected on different hardware, their times should not be interpreted as a controlled hardware-speed comparison.

H. Training and Validation Learning Curves

The tuned SVM is the validation-selected conventional classifier used for the confusion matrix, explainability, and ablation analyses. SVM training does not proceed through neural-network epochs and therefore does not produce epoch-wise loss and accuracy histories. BERT is the strongest epoch-trained contextual model in this study and is consequently used for the required learning-curve analysis.

Figure 4 presents the training and validation loss and accuracy produced by a diagnostic rerun of the selected BERT configuration. The same cleaned training pool, internal sampling procedure, pretrained revision, learning rate, batch size, and weight-decay settings are retained.

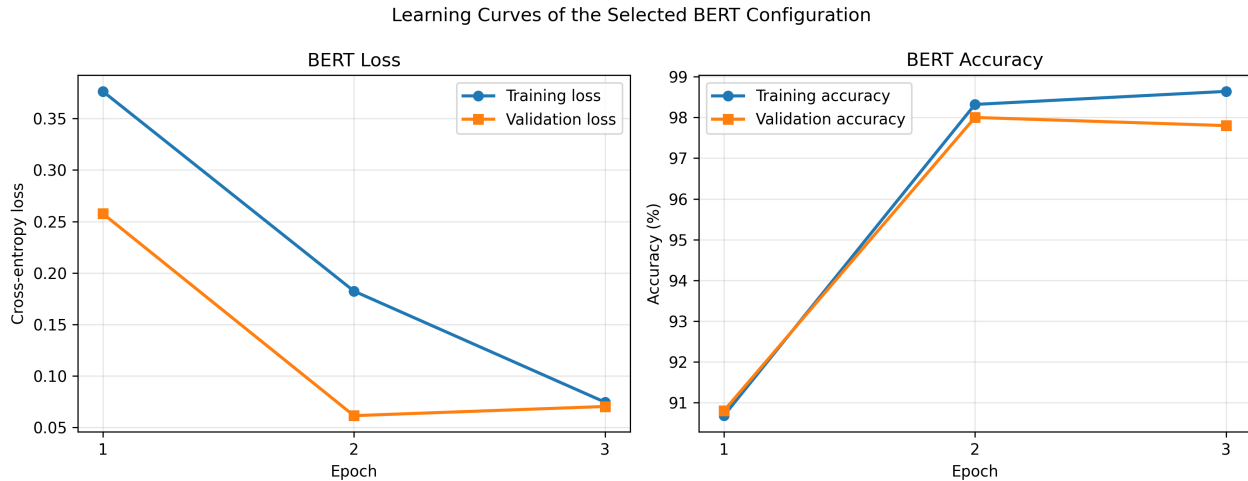


Fig. 4: Training and validation loss and accuracy of the selected BERT configuration over three epochs.

Validation accuracy increases from 90.80% to 98.00% in the first two epochs, then drops slightly to 97.80% in the third epoch. Training loss steadily decreases from 0.3764 to 0.0746. Validation loss also decreases from 0.2577 to 0.0615, then rises slightly to 0.0705.

This small difference shows a slight generalization gap. It also shows why the best validation checkpoint is better than simply using the final epoch. This check was only used to study the learning curve. It does not affect the main BERT test results.

I. Discussion of Shared Experiments

1) *Machine Learning and Ensemble Performance:* XGBoost performed best among the five baseline models, with 99.72% accuracy and 99.82% phishing F1-score. Random Forest and Decision Tree also performed well. This shows that URL features work well with tree-based models.

After tuning, SVM gave the best overall results, with 98.71% balanced accuracy and 99.09% phishing F1-score. Logistic Regression also gave balanced results.

Decision Tree, Random Forest, and XGBoost reached 100% phishing recall. However, their balanced accuracy dropped to 94.99% because more legitimate URLs were incorrectly flagged as malicious.

The three tree models made the same final predictions, but their ROC-AUC scores were different. Stacking also reached 100% phishing recall but did not beat SVM. Blending performed better than stacking on the main metrics. Its 98.69% F1-score remained slightly below the tuned SVM result of 99.09%. Therefore, combining multiple models did not automatically produce better performance than the strongest individual model.

2) *Effect of Data Augmentation:* Among the augmentation methods, SMOTE obtained the highest accuracy and F1-score, with 97.95% accuracy and 98.67% F1. Random oversampling gave a very similar result, while ADASYN produced the weakest performance of the three methods.

The tuned SVM without augmentation obtained 98.59% accuracy, 98.49% phishing recall, and 99.09% F1-score. Consequently, none of the three augmentation methods improved the unaugmented SVM result. The original training set already contained a large number of phishing samples, and balancing the legitimate minority class changed the decision boundary in a way that slightly reduced phishing recall.

3) *NLP Representation Performance:* Validation tuning improved all NLP models. TF-IDF accuracy increased from 89.46% to 94.54%. Word2Vec reached 91.43% accuracy. FastText performed best, with 95.27% accuracy and 96.92% F1-score.

However, tabular models performed better overall. The NLP models used serialized URL features instead of webpage content. This tested whether structured URL features could work as text.

TABLE XIV: Matched Random Forest Results Under Different Phishing-Label Budgets

Labeled Ratio	Acc. (%)	BA (%)	Precision (%)	Recall (%)	F1 (%)	AUC	AP
1%	96.52	97.61	99.87	95.65	97.72	0.9992	0.9998
5%	98.40	98.71	99.79	98.16	98.96	0.9994	0.9998
10%	99.13	99.14	99.76	99.12	99.44	0.9995	0.9998
25%	99.48	99.36	99.76	99.58	99.67	0.9995	0.9999
50%	99.60	99.44	99.76	99.73	99.74	0.9996	0.9999
100%	99.67	99.48	99.76	99.82	99.79	0.9996	0.9999

TABLE XV: StablePFN-Inspired Pipeline Performance

Model	Acc.	BA	Precision	Recall	F1	AUC
Stability Pipeline	99.29	98.74	99.37	99.73	99.55	0.9993

4) *Contextual and LLM Performance*: The results reached 97.81% accuracy and a 98.61% F1-score. It rivaled standard baselines but did not surpass XGBoost. DistilGPT-2, improved with tuning to 92.23% accuracy, yet trailed BERT. The bidirectional encoder in BERT proved far more capable with structured attribute text than an autoregressive decoder.

The zero-shot Qwen baseline classified all test samples as phishing. Because phishing samples constitute 77.92% of the test set, this produced 77.92% accuracy but only 50% balanced accuracy. After validation-based prompt selection, Qwen’s accuracy increased to 89.79%, while its phishing F1-score reached 93.12%. The selected prompt, therefore, produced a meaningful improvement over the zero-shot baseline.

The contextual results should be interpreted according to their different adaptation procedures. BERT and DistilGPT-2 were trained by updating model weights, whereas Qwen retained frozen weights and used a selected few-shot prompt. The comparison, therefore, evaluates three contextual implementation strategies on the same test set rather than three identically trained models.

5) *Summary of the Shared Experiments*: The experiments demonstrate that numerical tabular models remain highly effective for the available engineered URL features. XGBoost produced the strongest baseline result, while SVM gave the strongest balanced result among the validation-selected models. Stacking, blending, and augmentation did not surpass the strongest individual model on every metric. BERT was the strongest contextual model, and FastText was the strongest NLP representation.

J. Paper-Based Pipeline Results

This section presents the results of the three paper-based pipeline implementations. The WSADBench-inspired Random Forest and pretrained iLTM experiments use identical labeled-phishing ratios and the same held-out test set. This allows their results to be compared under the same label budgets.

1) *WSADBench-Inspired Label-Budget Results*: The WSADBench-inspired experiment evaluates how a fixed Random Forest reference model performs when only a selected proportion of the phishing training samples is included [18]. All 6,121 legitimate training samples are retained at every ratio. The unselected phishing samples are excluded from supervised training and are not treated as unlabeled data.

Table XIV shows that the Random Forest reference model improves as additional phishing labels are included. Even at the 1% ratio, corresponding to 214 labeled phishing samples, it obtains 97.72% F1. At 10%, its F1-score reaches 99.44%, which is 0.35 percentage points below the fully supervised result. Increasing the ratio from 25% to 100% produces comparatively small improvements.

These results show that this Random Forest can learn effectively from a limited phishing-label budget. This experiment is a supervised label-budget adaptation of the WSADBench idea. It is not a reproduction of the complete weakly supervised anomaly-detection benchmark.

2) *StablePFN-Inspired Pipeline Results*: The StablePFN-inspired experiment uses repeated Random Forest models to rank features according to the stability of their importance values [19]. The ten highest-ranked features are used to train an XGBoost classifier.

The accuracy, balanced accuracy, precision, recall, and F1 entries in Table XV are percentages. The pipeline obtains 99.29% accuracy and 99.55% phishing F1-score while using only ten selected features.

TABLE XVI: Pretrained iLTM Performance Under Different Phishing-Label Budgets

Labeled Ratio	Acc. (%)	BA (%)	Precision (%)	Recall (%)	F1 (%)	AUC	AP
1%	89.36	92.79	99.65	86.65	92.70	0.9917	0.9976
5%	93.32	95.60	99.90	91.51	95.52	0.9972	0.9992
10%	97.91	98.39	99.78	97.52	98.64	0.9989	0.9996
25%	99.20	99.33	99.88	99.09	99.48	0.9977	0.9989
50%	99.48	99.09	99.55	99.79	99.67	0.9983	0.9991
100%	99.79	99.63	99.82	99.91	99.86	0.9966	0.9976

TABLE XVII: iLTM F1 With and Without Retrieval

Ratio	Retrieval On	Retrieval Off	Difference
1%	92.70	97.83	-5.13
5%	95.52	95.86	-0.33
10%	98.64	98.78	-0.14
25%	99.48	99.48	0.00
50%	99.67	99.68	-0.01
100%	99.86	99.86	0.00

This experiment adapts the feature-stability principle associated with StablePFN. It does not execute the original pretrained StablePFN model. Therefore, the result should be interpreted as the performance of the implemented stability-based feature-selection pipeline.

3) *Pretrained iLTM Results:* The iLTM experiment uses the available pretrained iLTM checkpoint for downstream adaptation to the phishing dataset [6]. The selected checkpoint combines robust numerical preprocessing, XGBoost-derived representations, random-feature projection, a pretrained hypernetwork-generated MLP, and retrieval-augmented prediction.

The iLTM results improve substantially as the phishing-label budget increases. At the 1% ratio, the model obtains 92.70% F1, while the 10% ratio raises F1 to 98.64%. The fully supervised configuration achieves 99.79% accuracy and 99.86% F1. In that configuration, the model correctly detects 3,308 of the 3,311 phishing test samples, with three false negatives and six false positives.

The matched Random Forest performs better than iLTM at the 1%, 5%, 10%, 25%, and 50% label ratios. At the 100% ratio, iLTM exceeds the matched Random Forest by approximately 0.12 percentage points in accuracy and 0.08 percentage points in F1-score. Therefore, the pretrained iLTM implementation provides its strongest relative result when the complete training set is available.

This experiment performs downstream adaptation using an available pretrained iLTM checkpoint. It does not reproduce the original large-scale meta-training procedure used to create the pretrained model.

4) *iLTM Retrieval Ablation:* To examine the contribution of retrieval augmentation, predictions were also generated with retrieval disabled. This ablation uses the same trained iLTM weights and changes only the inference procedure.

All entries in Table XVII are F1 percentages, and The difference is calculated as retrieval-on minus retrieval-off. Retrieval did not improve the hard-label F1-score in this experiment. Its largest negative effect occurred at the 1% ratio. At the 25% and 100% ratios, enabling retrieval, did not change the final hard-label predictions.

This result does not show that retrieval is ineffective in every iLTM application. It shows that the selected retrieval settings did not provide an F1 improvement for this dataset and evaluation split. Further tuning of the retrieval weight, temperature, and reference set size would be required to determine whether a different configuration could produce a benefit.

K. Explainable Artificial Intelligence

LIME is used to generate a local explanation for a selected prediction from the tuned SVM [20]. The explainer is fitted using the 16 numerical features from the outer training pool and uses random seed 42.

The first record of the shared outer test set is selected as an illustrative example. LIME creates perturbed records around that sample and obtains probability outputs from the tuned SVM. A locally weighted interpretable model estimates how the feature conditions affect the explained class in the neighborhood of that record.

The saved explanation concerns **class 1, the legitimate class**. Therefore:

- a positive local weight supports the legitimate class score for the explained record; and
- a negative local weight opposes the legitimate-class score and locally supports the phishing alternative.

Table XVIII presents the ten saved local feature contributions.

TABLE XVIII: LIME explanation for the legitimate-class score of the first test record

Local feature condition	Local weight
<code>https_flag ≤ 0</code>	−0.0770
<code>subdomain_count ≤ 1</code>	−0.0432
<code>token_count ≤ 7</code>	+0.0422
<code>suspicious_file_extension ≤ 0</code>	+0.0286
<code>dot_count ≤ 3</code>	+0.0209
<code>tld_popularity ≤ 0</code>	−0.0196
<code>percentage_numeric_chars ≤ 0</code>	+0.0194
<code>has_ip_address ≤ 0</code>	+0.0189
<code>number_of_digits ≤ 0</code>	+0.0154
<code>has_hyphen_in_domain > 0</code>	+0.0140

For this local explanation, the absence of HTTPS has the largest absolute coefficient and moves the local explanation away from the legitimate class. A subdomain count no greater than one also has a negative coefficient. In contrast, a token count no greater than seven has the largest positive coefficient for the legitimate-class score.

These coefficients describe the fitted local surrogate around one test record. They are not class probabilities, global feature importance values, or causal effects. The saved output does not include the explanation’s local fidelity, the record’s predicted probability, or its true label. Therefore, no claim about those unavailable values is made.

The tuned SVM is used in this post hoc illustration. The LIME Example does not change model training, configuration selection or the reported test metrics.

L. Normalized Confusion-Matrix Analysis

The selected tuned SVM is further examined using the row-normalized confusion matrix in Fig. 5. Normalization is performed independently for each true class, allowing the class-wise classification rates to be compared despite the class imbalance in the test partition.

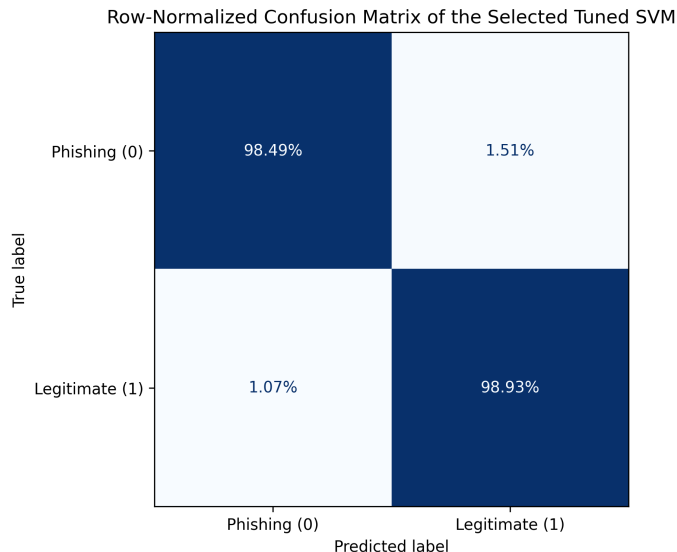


Fig. 5: Row-normalized confusion matrix of the selected tuned SVM on the shared outer test set.

The model correctly classifies 98.49% of the phishing records and 98.93% of the legitimate records. The phishing false-negative rate is 1.51%, representing phishing samples incorrectly classified as legitimate. The legitimate false-positive rate is 1.07%, representing legitimate samples incorrectly classified as phishing. These class-wise results are consistent with the model’s 98.71% balanced accuracy and show that its performance is not produced solely by the larger phishing class.

M. Ablation Study

Feature-group ablation is performed using the selected tuned SVM, which is also used for the LIME explanation and normalized confusion matrix. This maintains one consistent reference model for the best-model diagnostic experiments. For each ablation setting, a new SVM is fitted after removing the corresponding feature group from both the outer training and test partitions.

The security-indicator group contains the IP address, HTTPS, TLD-popularity and suspicious-file-extension attributes. The lexical complexity group contains URL entropy, percentage of numerical characters, and token count. The domain-structure group contains dot count, subdomain count, path length, and domain-name length.

TABLE XIX: Feature-Group Ablation Using the Selected Tuned SVM

Setting	Features	Acc. (%)	BA (%)	Prec. (%)	Recall (%)	F1 (%)	AUC
All features	16	98.59	98.71	99.69	98.49	99.09	0.9970
Without security indicators	12	98.35	96.31	97.96	99.97	98.95	0.9827
Without lexical complexity	13	98.40	98.78	99.85	98.10	98.96	0.9969
Without domain structure	12	97.20	97.25	99.23	97.16	98.18	0.9912

Removing the domain-structure features produces the largest reduction in accuracy and phishing F1. Accuracy decreases from 98.59% to 97.20%, while F1 decreases from 99.09% to 98.18%. Removing the security indicators produces the largest reduction in balanced accuracy and ROC-AUC, although phishing recall increases to 99.97%. The model predicts more URLs as phishing. This improves phishing sensitivity but lowers precision and legitimate-class performance.

Removing lexical-complexity features has a smaller effect. Balanced accuracy and precision increase slightly, while accuracy, recall, and F1-score decrease.

Different feature sets affect the metrics differently. Domain features mainly improve phishing F1-score. Security features help maintain class balance and improve threshold-independent results.

These results show feature dependence in the tested dataset. They do not show causal relationships.

N. External Validation and Target Adaptation

We also tested the best baseline model on the ISCX-URL2016 phishing dataset [21]. The two datasets use different feature schemas. So, we selected five compatible features: URL length, dot count, path length, digit count, and percentage of numeric characters.

The XGBoost model was retrained on the LegitPhish training set using these five features. Its hyperparameters were fixed before testing on ISCX.

The ISCX dataset initially had 15,367 records. After removing 548 conflicting records and 5,954 repeated feature rows, 8,865 records remained. This cleaned set contained 4,465 phishing and 4,400 benign samples. In the source-only external test, no ISCX record was used for fitting, tuning, or threshold selection.

TABLE XX: XGBoost External Validation and Target Adaptation on ISCX-URL2016

Training setting	Test set	Rows	Acc. (%)	BA (%)	Prec. (%)	Recall (%)	F1 (%)
Source only	Completely cleaned ISCX	8,865	61.99	61.72	57.15	98.01	72.20
Source only	Matched 90% ISCX test	7,979	61.95	61.68	57.13	97.98	72.18
Source plus 10% ISCX adaptation	Matched 90% ISCX test	7,979	91.10	91.09	90.17	92.41	91.28

The source-only model reached 61.99% accuracy and 72.20% phishing F1 on the completely cleaned ISCX dataset. Its high phishing recall of 98.01% was accompanied by many benign URLs being incorrectly classified as phishing. This result shows that the relationships between the five features and the class labels differ considerably between LegitPhish and ISCX.

A second experiment examined supervised target adaptation. A stratified 10% partition containing 886 labeled ISCX records (446 phishing and 440 benign) was combined with the LegitPhish training partition. The remaining 7,979 ISCX records were kept untouched for final testing. The XGBoost hyperparameters and 0.50 decision threshold were kept unchanged. The source and target training sets were given equal total weight. This allowed the smaller target set to still influence the model.

No records from the final 90% partition were used during fitting.

On the untouched 90% test partition, adaptation increased accuracy from 61.95% to 91.10% and phishing F1 from 72.18% to 91.28%. ROC-AUC increased from 0.8242 to 0.9673. Figure 6 compares the results before and after adaptation. This shows supervised adaptation to the target dataset. It should not be considered zero-shot generalization.

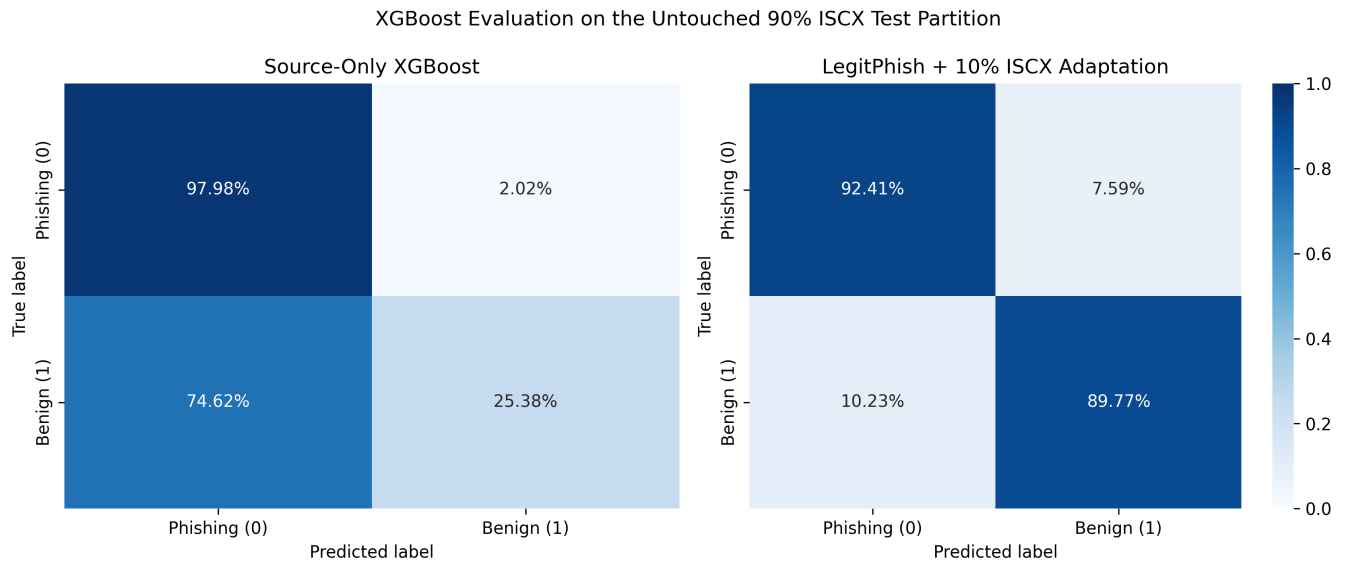


Fig. 6: Confusion matrices showing XGBoost results before and after 10% ISCX adaptation on the same 90% test set.

O. Comparison with Existing Studies

Table XXI contrasts our results against five existing phishing-detection studies. These detail the specific dataset and evaluation setup for each, accounting for varying experimental conditions.

TABLE XXI: Comparison with Existing Phishing-Detection Studies

Study and model	Dataset / evaluation	Acc. (%)	Prec. (%)	Recall (%)	F1 (%)
[4]: EGSO-CNN	Authors' phishing dataset and evaluation	99.44	–	99.21	99.32
[5]: LightGBM	Authors' balanced dataset; 10-fold cross-validation	97.28	–	96.02	–
[22]: Random Forest with NLP features	73,575 phishing and legitimate URLs	97.98	97.00	99.00	98.00
[23]: Random Forest with ranked features	11,430 websites; chi-square feature ranking	96.83	–	–	–
[24]: XGBoost	Average across three cross-dataset test settings	77.84	73.60	82.89	77.93
This work: selected tuned SVM	LegitPhish; shared outer test set of 4,249 records	98.59	99.69	98.49	99.09
This work: pretrained iLTM, 100% phishing-label budget	LegitPhish; shared outer test set of 4,249 records	99.79	99.82	99.91	99.86

Published values mirror the original metrics and testing methods chosen by their respective authors. Throughout our project, phishing is treated as the positive class. Note that a dash means a metric couldn't be confirmed from the source reference—it isn't a zero.

Because these studies rely on varying datasets, preprocessing workflows, and evaluation setups, the reported numbers offer descriptive context rather than a strict baseline ranking. Sahingoz et al., along with Hannousse and Yahiouche, demonstrated high performance within their own data splits. On the other hand, Misiak and Hyla's cross-dataset testing showed noticeable drop-offs, underscoring how out-of-domain evaluation impacts accuracy—a trend mirrored in our source-only ISCX tests. Lastly, on our joint LegitPhish test set, the fully trained iLTM outperformed our tuned SVM in both overall accuracy and phishing F1.

P. Limitations

Several limitations were found when interpreting these experimental results. Like:

- Primary model development relied on a single phishing URL dataset paired with one fixed train-test split. Although the split prevents the structural group overlap between training and testing, results from a single split do not measure variation across multiple random seeds or test samples.
- The models use sixteen engineered URL features. Their strong Results, therefore, depend on the quality and continued relevance of these features. Performance may decrease for newly developed phishing strategies that are not represented by the current feature set.
- The structural-group thresholds were calculated from the clean feature matrix before the outer split. Class labels and test outcomes were not used during this calculation, but future evaluations should estimate every preprocessing parameter using only the outer training data.
- Hyperparameter selection inside the outer training set used row-level validation or cross-validation. It was not consistently performed using the structural groups applied to the outer split. Consequently, internal validation estimates may be more optimistic than results from a completely group-aware nested evaluation.
- External validation was conducted on ISCX-URL2016 using only five attributes with compatible definitions. The ISCX file did not provide the complete sixteen-feature schema or raw URL strings. Consequently, the experiment required a five-feature refit, and exact URL overlap between the two datasets could not be checked.
- The 10% target-adaptation result uses labeled ISCX samples and Therefore, measures supervised adaptation rather than zero-shot transfer. It was evaluated using one fixed stratified split; additional Splits would be needed to estimate variation across different targets samples.
- The NLP and contextual models receive serialized versions of the engineered features. They examine full HTML, domain WHOIS records, or context messages. These models evaluate only serialized features.
- While BERT and DistilGPT-2 undergo full weight updates during training, Qwen remains frozen and relies on prompt tuning instead. This disparity in compute and training approaches prevents a direct apples-to-apples architectural comparison. Furthermore, Qwen's raw next-token output doesn't reflect true calibrated probabilities.
- The WSADBench experiment adapts only the labeled-anomaly-ratio concept. Unselected phishing samples are excluded instead of being used as unlabeled observations. It is therefore not a complete reproduction of the original weakly supervised anomaly-detection benchmark.
- The StablePFN-inspired experiment implements stability-based feature selection followed by XGBoost. This setup skips the original pretrained StablePFN model.
- Instead of repeating full-scale meta-training, we simply fine-tuned a pretrained iLTM checkpoint. Keeping retrieval adjustments minimal meant that retrieval offered no noticeable boost to the F1 score.
- Resource benchmarks reflect split runtimes across differing hardware: NLP baselines ran on a Google Colab CPU, while contextual models ran on an NVIDIA Tesla T4 GPU. These runtimes log actual trials. We also did not track power draw or full deployment latency.

TABLE XXII: Principal Software Dependencies

Purpose	Principal Packages
Data processing	numpy, pandas, scipy
Machine learning	scikit-learn, xgboost, imbalanced-learn, joblib
NLP representations	gensim, scikit-learn
Contextual models	torch, transformers, accelerate
Interpretability	lime
Visualization	matplotlib, seaborn
iLTM adaptation	Pretrained iLTM package and dependencies

Q. Source Code and Reproducibility

The executed notebooks and supporting result artifacts used in this Study are organized in the following repository:

<https://github.com/mahfuj142/CSE445-Phishing-URL-Detection>

The reproducibility package contains eight principal notebooks:

- `01_Shared_ML_NLP_Pipeline.ipynb`, containing the data preparation steps, the five machine-learning models, and the hyperparameter selection, stacking, blending, augmentation, NLP representations, explainability, feature ablation, and the paper-inspired reference experiments.
- `02_BERT_DistilGPT2_Qwen.ipynb`, containing the corrected BERT, DistilGPT-2, and Qwen experiments with validation-based configuration selection. This notebook supersedes the preliminary contextual-model cells in the shared notebook.
- `03_WSADBench_Pipeline_Prototype.ipynb`, containing the KDD 2026 WSADBench-inspired phishing URL detection pipeline, the LegitPhish dataset preprocessing and 16-feature engineering steps, The label-budget experiment across six labeled phishing ratios (1%, 5%, 10%, 25%, 50%, 100%), The Random Forest classifier training and evaluation (accuracy, precision, recall, F1, ROC-AUC), confusion matrix analysis, and feature importance extraction with exported result files. This notebook is retained as the original individual WSADBench-inspired prototype. Because it uses a random row split, its displayed metrics are not reported in this paper. The common-split Random Forest results in Table XIV is generated by Notebook 04 so that Random Forest and iLTM use identical label pools and the same outer test partition.
- `04_iLTM_Pipeline.ipynb`, containing the pretrained iLTM downstream-adaptation experiment, matched Random Forest comparison, label-budget evaluation, retrieval ablation, and exported result files.
- `05_CSE445_model_efficiency.ipynb`, containing the A supplementary diagnostic rerun is used to measure the NLP model parameter counts, approximate weight sizes, and training times under a fresh Colab CPU session. This notebook also reproduces the feature-group ablation and normalized confusion-matrix outputs for the tuned SVM.
- `06_BERT_training_and_validation_curves.ipynb`, containing the diagnostic rerun of the selected BERT configuration used to generate the epoch-wise training and validation loss and accuracy histories. The learning-curve figure reported in Fig. 4 and the exported `bert_learning_history.csv` are produced by this notebook.
- `07_ISCX_External_Validation_XGBoost_and_SVM.ipynb`, containing the source-only five-feature external evaluation on the cleaned ISCX-URL2016 dataset.
- `07b_ISCX_10pct_Target_Adaptation_XGBoost.ipynb`, containing the stratified 10% labeled ISCX target-adaptation experimentation and evaluation on the untouched remaining 90%.

The repository also contains the saved CSV result tables and evaluation artifacts used to prepare the report. Large pretrained checkpoints are not included and are downloaded by their corresponding notebook when required.

The exact installation commands and compatibility fixes are included at the beginning of the corresponding notebooks. Installing packages in a fresh Google Colab session is recommended because changing numerical libraries in an active runtime can produce binary-compatibility errors.

The conventional machine learning and NLP experiments can be executed using a CPU runtime. A GPU-enabled Google Colab runtime is recommended for BERT, DistilGPT-2, Qwen, and iLTM because these experiments require more memory and computation. Each notebook exports its results to a separate run directory so that outputs from different configurations are not combined.

IV. CONCLUSION

This project developed and evaluated a phishing URL detection framework covering conventional machine learning, ensemble learning, and data augmentation, NLP representations, contextual language models, explainability, feature ablation, and three paper-inspired pipelines. Evaluation was performed on a common feature-deduplicated outer split containing 27,559 training rows and 4,249 test rows, with no structural-group overlap between the two partitions.

Among the five baseline machine learning models, XGBoost produced the strongest result, achieving 99.72% accuracy and 99.82% phishing F1-score. After hyperparameter selection, SVM provided the strongest balance in the primary six-model comparison, with 98.71% balanced accuracy and 99.09% F1. Stacking and blending did not surpass the strongest individual model on all metrics. Similarly, none of the three augmentation methods improved the unaugmented tuned SVM result.

FastText was the strongest NLP representation, reaching 95.27% accuracy and 96.92% F1. BERT was the strongest contextual model, achieving 97.81% accuracy, 99.37% phishing recall, and 98.61% F1. DistilGPT-2 and Qwen improved after validation-based selection, although their results remained below BERT. These findings indicate that contextual models can classify serialized URL features effectively, but the strongest tabular models remain more suitable for the available engineered features.

The WSADBench-inspired Random Forest demonstrated strong performance under limited phishing-label availability. It achieved 97.72% F1 with 1% labeled phishing data and 99.44% F1 with 10%. The StablePFN-inspired stability pipeline achieved 99.29% accuracy and 99.55% F1 using ten selected features. The pretrained iLTM pipeline improved substantially as more phishing labels were provided, increasing from 92.70% F1 at 1% labels to 99.86% at full supervision. Its fully supervised configuration achieved the highest observed hard-label accuracy and F1-score, at 99.79% and 99.86%, respectively. The matched Random Forest was stronger than iLTM at the lower label ratios, and retrieval augmentation did not improve iLTM F1 in the reported configuration. Our interpretability and ablation studies provided further perspective on the model's behavior. The LIME inspection showed how individual attributes shifted a single SVM decision, while dropping the domain feature set caused the sharpest decline in tuned SVM accuracy and F1-score, reducing accuracy from 98.59% to 97.20% and phishing F1 from 99.09% to 98.18%. These patterns remain data-driven. So, they should not be viewed as causal rules.

The model's external test showed a clear shift between the source and target data distributions. Without fine-tuning, XGBoost reached just 61.99% accuracy on the cleaned ISCX-URL2016 dataset. However, adding just 10% labeled target data bumped performance to 91.10% accuracy and a 91.28% phishing F1 across the remaining 90%. These compared figures highlight the difference between out-of-the-box transfer and targeted supervised adaptation.

A. Future Work

The external evaluation showed that a model trained on one URL dataset may lose accuracy when it is applied to data collected from another source. A future experiment should therefore collect recent phishing URLs from services such as PhishTank and OpenPhish, together with an independent list of legitimate URLs. The same 16 features used in this project could then be extracted directly from those raw URLs, allowing the complete trained pipeline to be tested without reducing it to five shared features.

Further experiments should also use several target-data splits and different random seeds to determine whether the improvement from 10% labeled target adaptation remains consistent. A time-based test, where newer URLs are evaluated after training on older records, would provide a more realistic measure of performance against changing phishing strategies. The final model could later be connected to a small browser extension or web service, although its prediction time, memory use, and false-alarm rate would need to be tested before practical use.

REFERENCES

- [1] A. Aleroud and L. Zhou, "Phishing environments, techniques, and countermeasures: A survey," *Computers & Security*, vol. 68, pp. 160–196, 2017.
- [2] S. Sheng, M. Holbrook, P. Kumaraguru, L. F. Cranor, and J. Downs, "Who falls for phish? a demographic analysis of phishing susceptibility and effectiveness of interventions," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2010, pp. 373–382.
- [3] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Intelligent rule-based phishing websites classification," *IET Information Security*, vol. 8, no. 3, pp. 153–160, 2014.
- [4] K. Barik, S. Misra, and R. Mohan, "Web-based phishing URL detection model using deep learning optimization techniques," *International Journal of Data Science and Analytics*, vol. 20, no. 5, pp. 4449–4471, 2025.
- [5] J. D. Duarte, P. Chagas Junior, J. P. J. da Costa, E. J. da Costa, L. P. de Melo, R. R. Nunes, C. V. N. G. Soares, and T. E. da Cunha Silva, "Machine learning for early detection of phishing URLs in parked domains: An approach applied to a financial institution," *IEEE Access*, vol. 13, pp. 145 736–145 753, 2025.
- [6] D. Bonet, M. Comajoan Cara, A. Calafell, D. Mas Montserrat, and A. G. Ioannidis, "iLTM: Integrated large tabular model," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026, pp. 186–197.
- [7] R. Potpelwar, U. Kulkarni, and J. Waghmare, "Legitphish dataset," Mendeley Data, Version 2, 2025.
- [8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [11] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proceedings of the IEEE International Joint Conference on Neural Networks*, 2008, pp. 1322–1328.
- [12] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [13] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.
- [14] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019, pp. 4171–4186.
- [16] Hugging Face, "DistilGPT2 model card," <https://huggingface.co/distilbert/distilgpt2>, accessed September 4, 2026.

- [17] A. Yang *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
- [18] X. Yao, S. Zhou, Z. Wu, C. Hou, S. Liang, S. Wang, H. Huang, S. Han, and M. Jiang, “Rethinking weak supervision in anomaly detection: A comprehensive benchmark,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026, pp. 10 222–10 233.
- [19] Z. Guan, Y. Chen, H. Qian, K. Han, P. Cui, F. Wu, and K. Kuang, “StablePFN: Stable prediction with causal-aware tabular foundation model,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026, pp. 1381–1392.
- [20] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should i trust you? explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [21] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, “Detecting malicious urls using lexical analysis,” in *Network and System Security*, ser. Lecture Notes in Computer Science, vol. 9955. Springer, 2016, pp. 467–482.
- [22] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from urls,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [23] A. Hannousse and S. Yahiouche, “Towards benchmark datasets for machine learning based website phishing detection: An experimental study,” *Engineering Applications of Artificial Intelligence*, vol. 104, p. 104347, 2021.
- [24] M. Misiek and T. Hyla, “XGBoost-based url phishing detection method with cross-dataset validation,” *IEEE Access*, vol. 14, pp. 43 200–43 212, 2026.